Why we moved

Tunable NM-ROM — the decoder story from May to November 2026: what we tried, what broke, and why each step led to the next. · Tahmid Awal · November 3, 2026

We want to solve families of PDEs (heat, Poisson, Burgers, Navier–Stokes) fast, by solving for a **handful of numbers** instead of one unknown per mesh point. A **decoder** turns those few numbers into the full field, and the reduced solver adjusts them until the field satisfies the equations.

Every change below was an answer to one question: **which decoder gives a solve that is accurate *and* cheap?** Each time, the answer exposed a new problem, and that problem decided the next step.

Grid tensor (CP) → Attention → SIREN → POD fallback → FiLM + real solve → Bank + head → Ordered bank → Better solver, honest baselines → Obstacle flow

> **The one lesson that repeats:** a decoder that *reconstructs* a field well is not necessarily a decoder that *solves* well. Almost every dead end looked good as an autoencoder and failed inside the solver, or was accurate but too slow inside it.

1. The starting point

MAY 2026 · ORIGINAL HEAT AND POISSON MODELS

Grid tensor decoder (CP and LinearCP)

WHY WE TRIED IT

A small network picks weights for products of 1D factor tables (one per axis). Evaluating the field at a few selected points is very cheap, which is exactly what hyper-reduction needs. LinearCP added a linear "skip" so Poisson could start the solve from zero.

WHAT WENT WRONG

The factor tables are tied to one grid. Accuracy got **worse as the mesh got finer** — the "resolution wall" — and the skip connection did not fix it. Later we also found the public heat package had a bug that froze the rollout after step 1; with the fix, its published 39.6× speedup did not reproduce.

WHY WE MOVED

The bottleneck was the **spatial representation**, not the network that picks weights. We needed spatial functions that are not tied to a grid.

MAY 2026

Attention decoders (mirrored ViT, cross-attention)

WHY WE TRIED IT More expressive output maps, matching the ViT encoder.

WHAT WENT WRONG Reconstruction was competitive, but Poisson solves failed and heat rollouts became unusable. Self-attention over all output points destroys the cheap "evaluate only a few points" structure. Cross-attention needs tokens from a known solution snapshot — which, at solve time, is exactly what we don't have.

WHY WE MOVED We needed a decoder that can be queried **at any single point, independently**: a coordinate network.

MAY 2026

Modulated SIREN (coordinate network)

WHY WE TRIED IT Coordinates go in, the field value comes out; the latent code modulates the layers. Not tied to any grid, and the spatial features can change with the state.

WHAT WENT WRONG **The key failure of the project.** Accurate autoencoders still gave bad cold-start solves. Probes found a spurious "attraction basin" near the starting point, not a rank problem. Fixes (anchoring the zero code to a typical field) worked, then broke again when the network was made bigger or more accurate. And the whole network had to be evaluated and differentiated in *every* solver iteration — expensive.

WHY WE MOVED One variant made the output **affine in the latent code**, $u(x;z) = \Phi(x)Az + b(x)$. The solve became much easier. That was the first hint of the idea that eventually won: keep the spatial part fixed and simple online.

JULY 2026

Frozen POD, and POD + a residual network

WHY WE TRIED IT When nonlinear decoders gave unreliable rollouts, we fell back to a fixed linear basis from data (POD), optionally plus a small network to correct its error.

WHAT WENT WRONG Nothing broke — the surprise was that **POD is a very strong competitor**, once it is tuned and hyper-reduced fairly. The POD + network correction gave mixed gains.

WHY WE MOVED To beat POD we needed something POD cannot do: evaluate anywhere and transfer across meshes. Back to coordinate networks — this time with a real PDE solve.

2. Coordinate networks inside a real solve

AUGUST 2026

FiLM coordinate decoders, then latent stepping

WHY WE TRIED IT

FiLM lets the latent code scale and shift features throughout the network. It represented the families well and transferred across meshes. Latent stepping then made it a *true* ROM: the code is found from the PDE, not handed to the decoder.

WHAT WENT WRONG

The early wins had been fed the **true family parameters** — a surrogate, not a ROM. Once we solved for the code, hidden problems appeared: pointwise residuals amplified tiny grid-scale errors (fixed by smooth weak tests), and better reconstruction again did not mean better solves. Grouped FiLM reached about 0.5 % (Poisson) and 1 % (Burgers) mean error, but the network still ran inside every iteration, and fast kernels did not make the whole solve fast.

WHY WE MOVED

The cost lived in evaluating the coordinate network inside the inner loop. So: **take it out of the loop**.

3. The architecture that stuck

LATE AUGUST – SEPTEMBER 2026

Cached learned bank + small head

WHY WE TRIED IT

Split the decoder into a spatial **bank** of learned functions $g_a(x)$ and a small **head** $h_a(z)$: $u(x;z) = \Sigma_a\, g_a(x)\, h_a(z)$. Evaluate the bank once, cache it, and only the tiny head runs online.

WHAT WENT WRONG

This worked and became the core architecture. It also let us measure **bank error, head error and solve error separately** — and the head was the weak link. Every head variant we tried failed its accuracy gate (same frozen bank, matched controls):

HEAD	MEAN ERROR	WORST ERROR	OUTCOME
MLP (control)	5.5 %	17.0 %	baseline
Quadratic	4.7 %	20.3 %	better mean, worse worst case
Shared encoder	5.4 %	16.2 %	inconsistent across repeats
Mixture of local heads	5.5 %	21.3 %	no validation gain
Protected anchor	6.1 %	17.7 %	geometry fine, accuracy failed

WHY WE MOVED

A nonlinear head inside a fixed bank **can never leave that bank's span**. Making the head fancier was polishing the wrong part. Better to use the bank directly.

SEPTEMBER – OCTOBER 2026 · CURRENT BURGERS METHOD

Corrections, then the ordered bank

WHY WE TRIED IT

First, add linear correction directions on top of the head — accuracy could be raised after training, without retraining. In linear problems, solving with the full bank was both more accurate and cheaper than going through the head. So we ordered the bank once and solve directly for its first r coefficients: $u(x;c) = B_r(x)\, c$.

WHAT WENT WRONG

Nothing failed here — but it changed what the method *is*. It is linear in its online unknowns (neural only offline), which puts it head-to-head with tuned POD, which uses the same tricks.

WHAT IT GAVE US

A **deployment-time knob**: pick r for the accuracy/cost you want, with no retraining. This is the main Burgers path today.

SEPTEMBER – OCTOBER 2026 · NAVIER–STOKES SIDE BRANCH

Fixed frame → moving frame → divergence-free bank

WHY WE TRIED IT

Carry the bank idea to incompressible flow.

WHAT WENT WRONG

In a fixed frame, the bank + head **did not beat POD**. A divergence-free bank (curl of a neural potential) came close to its own floor but stayed less accurate than POD.

WHAT WORKED

Moving frames. Solving for a shift plus a shape, $u(x) = \tilde{u}(x - c; z)$, made translating flows compact — and here a nonlinear head genuinely helped. Our clearest win, but only for translating structures.

4. October: better solvers, honest comparisons

OCTOBER 2026 · JCP BURGERS CAMPAIGN AND BURGERS 3D REFINEMENT

Wider/smoother banks, off-mesh quadrature, second-order time stepping

WHY WE TRIED IT

Push accuracy and cost as far as possible before writing the paper, against much stronger, tuned baselines (spectral solvers, coarse grids, hyper-reduced POD).

WHAT WENT WRONG

Wider and smoother banks gave **better fits, not better solves**. The big accuracy gains came from the *solver* — off-mesh quadrature and better time stepping — not from the decoder. And the honest baselines removed the speed story: in Burgers 3D, every bank setting had a faster spectral solver at matching accuracy (bank took 1.5× to 176× as long). Continuous POD could also be reused across meshes, which withdrew our multi-mesh cost advantage.

WHY WE MOVED

What survives: **reduced cost that does not grow with the output mesh**, and evaluation at any point. What does not: a general speed or accuracy win over tuned spectral/POD on the box. So we stopped chasing speed on Burgers and looked for problems where neural banks could have a structural edge.

LATE OCTOBER 2026

Coarse-to-fine (multilevel) starts, and reconstruction vs interpolation

WHY WE TRIED IT

Could a cheap small-bank solve give a head start to the expensive fine solve? And does the bank reconstruct from samples better than classical interpolation?

WHAT WENT WRONG

Every coarse-to-fine variant was **slower** than just solving the fine problem directly (median, slower in every paired timing):

VARIANT	TIME VS DIRECT FINE SOLVE
Ordered-bank prefix start (affine / pure prefix)	1.39× / 1.44×
Separate small decoder (transfer / with physics)	2.62× / 3.38×
One shared 16/32/64 decoder (per step / replay)	1.41× / 1.87×

Reconstruction also missed both gates: the bank's error was 1.39× (2D) and 1.22× (3D) that of the best classical method, winning 0 of 32 and 2 of 64 cases.

WHY WE MOVED

The fine solve was already warm-started well, so there was no expensive fine work for a coarse level to remove. We stopped these lines unless someone has a concrete reason a new variant would cut fine-level work.

5. Where we are now

LATE OCTOBER – NOVEMBER 2026

Flow past obstacles (cylinder)

WHY WE'RE HERE

Complex geometry is where fast spectral solvers do not apply, and where a mesh-free neural bank might have a real edge over the alternatives.

WHAT WENT WRONG

The first trained bank failed badly (18–21 % velocity error, 36–44 % drag error). Several retraining rounds (trace-, energy- and traction-weighted fits, residual heads) each fixed some quantities and broke others — divergence and lift were the hardest.

STATUS

The latest neural-bank steady solves **pass all seven original development targets** (velocity error ≈ 0.036 %). Final validation and speed comparisons are still open. On Nov 3 we started the **unsteady cylinder** (vortex shedding, Re ≈ 100), an exactly divergence-free bank, a varying-geometry family, and an NS 3D test of whether a trained bank can match POD-64.

> **Proposed next architecture — global + local tree bank.** Keep a coarse global bank and add small local banks only where the error is. It is untested; we will first check with simple local POD or polynomial bases whether localizing helps at all, before training any neural features.

What the whole story taught us

- **Reconstruction ≠ solve.** Low autoencoder error says little about cold-start solves or stable time stepping.
- **Keep the expensive part out of the inner loop.** Caching the bank was the decisive design move.
- **Measure the bank, the head and the solve separately.** It shows which part actually needs work — usually not the one we were tuning.
- **Bigger or better-trained is not better-solved.** More features, width or training repeatedly improved fits without improving solutions.
- **The solver matters as much as the decoder.** Quadrature and time stepping gave the largest October gains.
- **Tuned classical baselines change the story.** POD, coarse grids and spectral solvers overturned several apparent wins; compare at matched accuracy and output.
- **Wins are real but scoped:** mesh-independent cost, point queries, moving frames for translating flows.

Sources: decoder-history report and collaborator PDF (Oct 28–29), JCP Burgers campaign final report, Burgers 3D refinement report, same-sample reconstruction report, and the project lab log through Nov 3, 2026. Numbers are copied from those records; full tables, scope qualifiers and retractions are in *Decoder-History-Collaborators.pdf*.